

SIMATS ENGINEERING

ASSESSMENT TOOL 1

**COURSE NAME: Artificial Intelligence
CSA17**

COURSE CODE:

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 50%

QUESTIONS: 5

Annexure A

**Duration: 1 Hour
Total Marks:50**

Design Task

Code of Conduct:

*I Shaik Arshad Basha (192312336) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Sk. Arshad Basha

Design Task

AI-Based Healthcare Monitoring and Alert System

Design a distributed AI healthcare monitoring system that uses PySpark and RDD operations to process continuous vital-sign data streams from thousands of wearable devices. Explain how RDD transformations support large-scale anomaly detection across the patient population, and describe how intelligent agents coordinate to generate early-warning alerts, prioritize critical cases for clinicians, and adapt monitoring thresholds to individual patient baselines.

Introduction

The AI-Based Healthcare Monitoring and Alert System is a distributed healthcare platform designed to continuously monitor vital signs collected from thousands of wearable devices. The system uses Apache Spark and RDD (Resilient Distributed Dataset) operations to process large volumes of patient data in parallel.

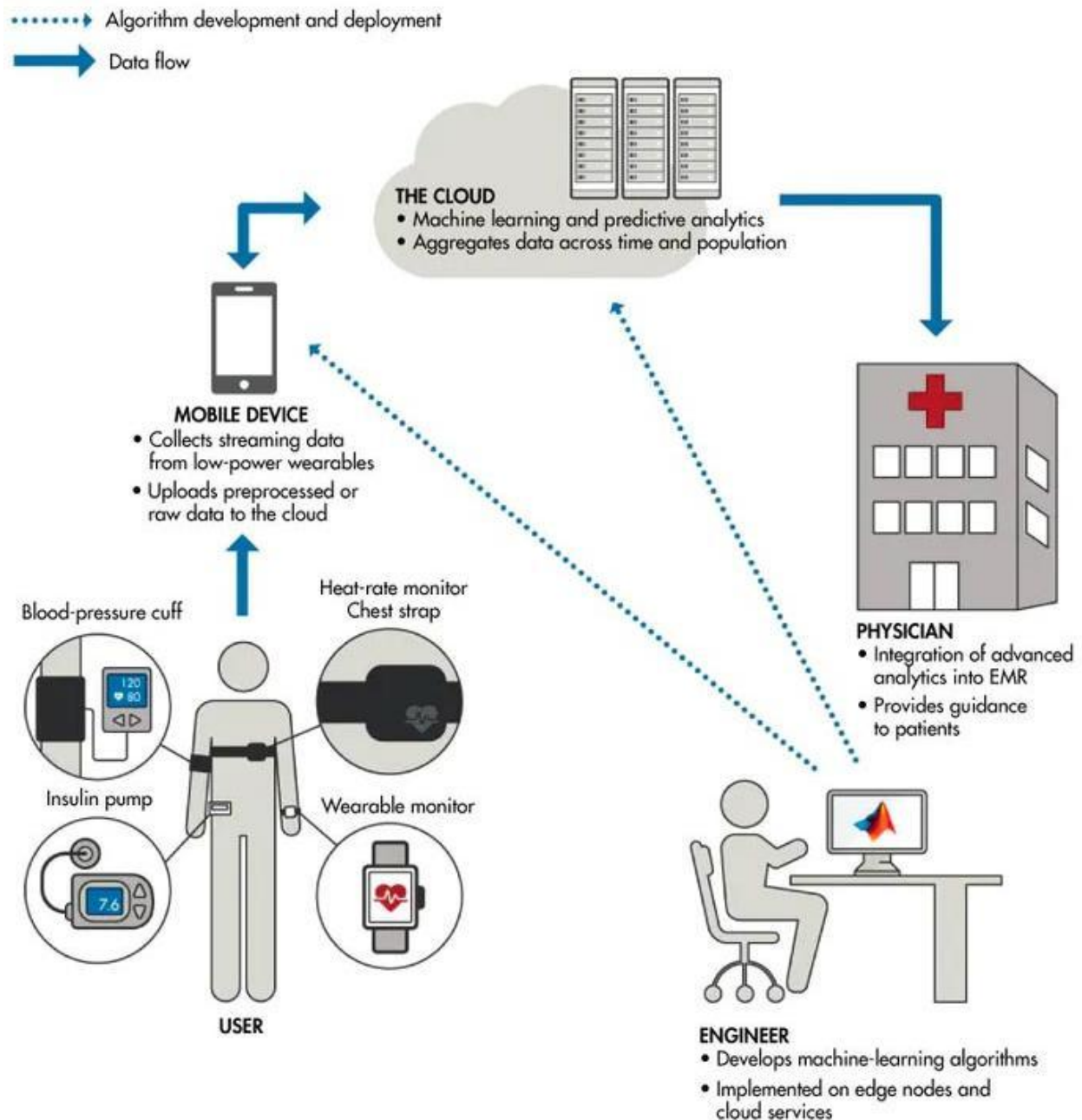
The system detects abnormal conditions such as high heart rate, low oxygen saturation, abnormal temperature, and irregular blood pressure, then uses intelligent agents to evaluate the severity and generate appropriate alerts for clinicians.

Main Objective

- To continuously monitor patients' vital signs using wearable devices.
- To use PySpark RDDs for scalable, distributed processing of healthcare data.
- To detect abnormal or dangerous vital-sign patterns in real time.
- To use intelligent agents to prioritize critical patients and generate early-warning alerts.

- To learn individual patient baselines and adapt monitoring thresholds accordingly.

3. System Architecture



Major Components

1. Wearable Devices

- Heart-rate sensor
- SpO₂ sensor
- Temperature sensor
- Blood-pressure sensor
- ECG sensor

2. Data Collection Layer

- Collects continuous vital-sign readings.
- Each reading contains:
 - Patient ID
 - Timestamp
 - Heart rate
 - SpO₂
 - Temperature
 - Blood pressure
 - ECG information

3. Distributed Processing Layer

- Uses **PySpark** to distribute data processing across multiple machines.
- RDDs divide large datasets into partitions that can be processed simultaneously.

4. AI Anomaly Detection Layer

- Identifies abnormal vital-sign values.

- Compares current readings with normal population ranges and individual patient baselines.

5. Intelligent Agent Layer

- Evaluates detected anomalies.
- Determines severity.
- Prioritizes patients.
- Generates alerts.

6. Clinician Dashboard

- Displays critical patients.
- Shows current vital signs and trends.
- Provides early-warning notifications.

Role of PySpark and RDD

RDD (Resilient Distributed Dataset) is a distributed collection of data that can be processed in parallel across Spark nodes.

For thousands of wearable devices, the data volume can become extremely large. Instead of processing every patient's data on one computer, Spark divides the data into partitions.

Example

Suppose 10,000 wearable devices generate:

$$10,000 \times 60 \text{ readings/minute} = 600,000 \text{ readings/minute}$$

Spark can distribute these readings across multiple worker nodes.

Alert Prioritization

Not every anomaly requires the same response.

The system calculates a **risk score** using factors such as:

- Severity of vital-sign abnormality
- Duration of abnormality
- Number of abnormal vital signs
- Patient's historical baseline
- Age/risk category where available
- Trend of the vital sign

Example:

Risk Score = Severity + Duration + Baseline Deviation + Multi-Signal Risk

Priority Levels

Priority 1 – Critical

- Severe abnormality
- Immediate clinician attention

Priority 2 – High

- Significant abnormality
- Rapid clinical review

Priority 3 – Medium

- Moderate abnormality

- Continued monitoring

Priority 4 – Low

- Minor deviation
- Store for later analysis

Advantages

- **Scalability:** Handles data from thousands of wearable devices.
- **Parallel Processing:** RDD partitions allow multiple patient records to be processed simultaneously.
- **Fault Tolerance:** RDDs can recover lost partitions through lineage.
- **Real-Time Detection:** Abnormal vital signs can be identified quickly.
- **Personalized Monitoring:** Thresholds can adapt to individual patient baselines.
- **Intelligent Prioritization:** Critical cases receive higher priority.
- **Early Warning:** Clinicians can be notified before a patient's condition becomes severe.

Result

The proposed system continuously processes large-scale healthcare data using PySpark RDDs and identifies abnormal vital-sign patterns efficiently. Intelligent agents analyze these anomalies, calculate risk levels, prioritize critical patients, and generate early-warning alerts. By combining population-level detection with patient-specific baselines, the system provides scalable, personalized, and proactive healthcare monitoring.

Conclusion

- PySpark RDD operations provide the distributed processing capability required to analyze continuous vital-sign data from thousands of wearable devices efficiently.
- Intelligent agents transform detected anomalies into prioritized early-warning alerts while adaptive patient baselines enable personalized and proactive monitoring.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations /actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration

Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined
Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation